

VIBE ID -- Complete Code Learning Guide

Line-by-Line Code Explanation

> This guide walks through every file and every important line of code in the VIBE ID project -- an AI-powered attendance system built with Streamlit, Supabase, dlib, and Resemblyzer. By the end, you should understand exactly how the entire codebase works and why each piece exists.

[Folder] Project Structure Overview

SnapClassSehaj/	
app.py	Main entry point
src/	
database/	
config.py	Supabase client setup
db.py	All database CRUD operations
pipelines/	
face_pipeline.py	Face recognition engine (dlib + SVM)
voice_pipeline.py	Voice recognition engine (Resemblyzer)
screens/	
home_screen.py	Landing page (role selection)
student_screen.py	Student login + dashboard
teacher_screen.py	Teacher login + dashboard + attendance
components/	
snapheader.py	Hero header with CSS animations
footer.py	Footer components
subject_card.py	Reusable subject card widget
dialog_add_photo.py	Camera/upload photo dialog
dialog_attendance_results.py	Attendance review & confirm
dialog_auto_enroll.py	QR-code auto enrollment
dialog_create_subject.py	Subject creation form
dialog_enroll.py	Manual enrollment by code
dialog_share_subject.py	QR code generation
dialog_voice_attendance.py	Voice attendance recording
ui/	
base_layout.py	Themes, CSS animations, particle canvas

1. app.py -- Main Entry Point (36 lines)

> [Pin] This is the file that runs when you type streamlit run app.py. It's the "front door" of the entire application.

```
import streamlit as st
```

Explanation: Imports the Streamlit library -- the web framework used to build the entire UI. Streamlit turns Python scripts into interactive web apps. Every st. call you see throughout the codebase comes from this import.

```
from src.screens.home_screen import home_screen
from src.screens.teacher_screen import teacher_screen
from src.screens.student_screen import student_screen
```

Explanation: Imports the three main screen functions -- one for each "page" of the app. Each function, when called, renders an entire page. This is how the app is organized: one function = one screen.

```
from src.components.dialog_auto_enroll import auto_enroll_dialog
```

Explanation: Imports the auto-enrollment dialog that opens when a student clicks a QR code join link (a URL with ?join-code=CS101). This needs to be at the app level because URL parameters are read globally.

```
def main():
    st.set_page_config(
        page_title='VIBE ID - Making Attendance faster using AI',
        page_icon= "https://i.ibb.co/YTYGn5qV/logo.png"
    )
```

Explanation: main() is the entry function for the whole app. st.set_page_config() sets the browser tab title and favicon (the small icon in the browser tab). [Key] This MUST be the first Streamlit command called -- if you call any other st. function before this, Streamlit will throw an error.

```
if 'login_type' not in st.session_state:
    st.session_state['login_type'] = None
```

Explanation: Initializes login_type in Streamlit's session state. Session state is a dictionary that persists data across page reruns. Every time a user interacts with the app (clicks a button, types text), Streamlit reruns the entire script from top to bottom. Without session state, all variables would be lost on each rerun. None means no role has been selected yet (show the home page).

```
match st.session_state['login_type']:
    case 'teacher':
        teacher_screen()
    case 'student':
        student_screen()
    case None:
        home_screen()
```

Explanation: Python's match-case statement (similar to switch-case in other languages, available in Python 3.10+). This is the app's router -- it checks what role the user selected and shows the appropriate screen. Since Streamlit doesn't have built-in page routing like React or Flutter, this pattern-matching approach serves as a manual page navigation system.

```
join_code = st.query_params.get('join-code')
if join_code:
    if st.session_state.login_type != 'student':
        st.session_state.login_type = 'student'
```

```
st.rerun()
if st.session_state.get('is_logged_in') and st.session_state.get('user_role') == 'student':
    auto_enroll_dialog(join_code)
```

Explanation: This handles QR code deep-linking. When a student scans a QR code, it opens a URL like `https://app.streamlit.app/?join-code=CS101`. This code:

- Reads the join-code parameter from the URL using `st.query_params.get()`
- If the user isn't on the student screen, forces them there with `st.rerun()`
- If the student is already logged in, opens the auto-enrollment dialog

[Bulb] `st.rerun()` restarts the script from the top with the updated session state.

```
main()
```

Explanation: Calls the main function to actually start the app. Without this line, nothing happens -- the function is defined but never executed.

2. src/database/config.py -- Supabase Client Setup (9 lines)

> [Pin] This tiny file creates the single database connection that every other file uses.

```
import streamlit as st
from supabase import create_client, Client
```

Explanation: Imports Streamlit (to access secrets) and the Supabase Python SDK. `create_client` is the function that establishes a connection, and `Client` is the type annotation for type-hinting.

```
supabase: Client = create_client(
    st.secrets["SUPABASE_URL"],
    st.secrets["SUPABASE_KEY"]
)
```

Explanation: Creates a Supabase client connection using the project URL and API key. These credentials are stored in `.streamlit/secrets.toml` -- a special file Streamlit reads automatically. `st.secrets` accesses values from that file securely, so credentials are never hardcoded in the source code.

[Key] This supabase variable is imported by every other file that needs database access. It's essentially a singleton -- one shared connection for the whole app.

3. src/database/db.py -- All Database Operations (98 lines)

> [Pin] This is the data layer -- every interaction with the database goes through a function in this file. Think of it as the

"API" for your database.

```
from src.database.config import supabase
import bcrypt
```

Explanation: Imports the Supabase client created in config.py and bcrypt -- a password hashing library. Bcrypt is industry-standard for securely storing passwords.

[Lock] Password Hashing Functions

```
def hash_pass(pwd):
    return bcrypt.hashpw(pwd.encode(), bcrypt.gensalt()).decode()
```

Explanation: Takes a plain-text password, performs three operations:

- `.encode()` -- converts the string to bytes (bcrypt works with bytes, not strings)
- `bcrypt.gensalt()` -- generates a random salt (random data added to the password before hashing to prevent rainbow table attacks)
- `bcrypt.hashpw()` -- hashes the password+salt combination
- `.decode()` -- converts the resulting bytes back to a string for database storage

[Key] Bcrypt is a one-way hash -- you can never reverse it to get the original password. That's the point.

```
def check_pass(pwd, hashed):
    return bcrypt.checkpw(pwd.encode(), hashed.encode())
```

Explanation: Compares a plain-text password (what the user just typed) against a stored bcrypt hash (from the database). Returns True if they match. The magic is that bcrypt extracts the salt from the stored hash itself (it's embedded in the hash string), re-hashes the plain-text password with that same salt, and compares the results.

[School] Teacher Functions

```
def check_teacher_exists(username):
    response = supabase.table("teachers").select("username").eq("username", username).execute()
    return len(response.data) > 0
```

Explanation: Queries the teachers table to check if a username already exists. Used during registration to prevent duplicate accounts. Breaking down the Supabase chain:

- `.table("teachers")` -- selects which table to query (like FROM teachers in SQL)
 - `.select("username")` -- which columns to return (like SELECT username)
 - `.eq("username", username)` -- filter condition (like WHERE username = 'value')
 - `.execute()` -- actually runs the query
 - `len(response.data) > 0` -- if any rows came back, the username exists
-

```
def create_teacher(username, password, name):
    data = { "username" : username, "password": hash_pass(password), "name": name}
    response = supabase.table("teachers").insert(data).execute()
    return response.data
```

Explanation: Creates a new teacher record. [Gear] Notice that the password is hashed with `hash_pass()` before being stored -- the plain-text password never touches the database. `.insert(data)` is Supabase's equivalent of SQL's `INSERT INTO`.

```
def teacher_login(username, password):
    response = supabase.table("teachers").select("*").eq("username", username).execute()
    if response.data:
        teacher = response.data[0]
        if check_pass(password, teacher['password']):
            return teacher
    return None
```

Explanation: The login flow:

- Fetch the teacher record by username (`.select("*")` gets all columns)
- If a record exists, extract it (`response.data[0]` -- first result)
- Verify the typed password against the stored hash using `check_pass()`
- If valid, return the full teacher record (for storing in session state)
- If invalid or no record found, return `None`

[Bulb] We never compare passwords directly -- always through `bcrypt`'s comparison function.

[Grad] Student Functions

```
def get_all_students():
    response = supabase.table('students').select("*").execute()
    return response.data
```

Explanation: Fetches every student record from the database. Used by the face recognition pipeline to get all stored face embeddings for training the classifier.

```
def create_student(new_name, face_embedding=None, voice_embedding=None):
    data = {'name': new_name, 'face_embedding': face_embedding, "voice_embedding": voice_embedding}
    response = supabase.table('students').insert(data).execute()
    return response.data
```

Explanation: Creates a new student with their name, face embedding (128-dimensional vector from face recognition), and optionally a voice embedding. The embeddings are stored as JSON arrays in Supabase. Default values are `None` because a student might register with only face data initially.

Subject Functions

```
def create_subject(subject_code, name, section, teacher_id):
    data = {"subject_code": subject_code, "name": name, "section": section, "teacher_id": teacher_id}
    response = supabase.table("subjects").insert(data).execute()
    return response.data
```

Explanation: Creates a new subject/course. teacher_id links it to the teacher who created it -- this is a foreign key relationship.

```
def get_teacher_subjects(teacher_id):
    response = supabase.table('subjects').select(
        "*", subject_students(count), attendance_logs(timestamp)"
    ).eq("teacher_id", teacher_id).execute()
    subjects = response.data
```

Explanation: This is the most complex query in the app. It fetches all subjects for a teacher along with related data from two other tables:

- subject_students(count) -- counts how many students are enrolled (Supabase aggregate)
- attendance_logs(timestamp) -- fetches all attendance timestamps

This is a Supabase join -- instead of writing separate queries, you embed related table data in a single request.

```
for sub in subjects:
    sub['total_students'] = sub.get("subject_students", [{}])[0].get('count', 0) if sub.get('subject_stud
```

Explanation: Extracts the student count from the nested Supabase response. The count comes back as [{"count": 5}], so we access index [0] then the 'count' key. The chain of .get() calls with defaults prevents crashes if data is missing.

```
attendance = sub.get('attendance_logs', [])
unique_sessions = len(set(log['timestamp'] for log in attendance))
sub['total_classes'] = unique_sessions
```

Explanation: Calculates how many unique class sessions have been held. Multiple students can have the same timestamp (they were all marked in the same session), so we use set() to deduplicate. The number of unique timestamps = number of classes held.

```
sub.pop('subject_student', None)
sub.pop('attendance_logs', None)
return subjects
```

Explanation: Cleans up the response by removing the raw nested data (we've already extracted what we need into total_students and total_classes). .pop(key, None) removes a key if it exists, does nothing if it doesn't.

Enrollment Functions

```
def enroll_student_to_subject(student_id, subject_id):
    data = {'student_id': student_id, "subject_id": subject_id}
    response = supabase.table('subject_students').insert(data).execute()
    return response.data
```

Explanation: Creates a record in the junction table subject_students. This is a many-to-many relationship -- one student can be in many subjects, one subject can have many students. The junction table links them.

```
def unenroll_student_to_subject(student_id, subject_id):
    response = supabase.table('subject_students').delete().eq(
        'student_id', student_id
    ).eq('subject_id', subject_id).execute()
    return response.data
```

Explanation: Removes the enrollment record. `.delete()` with two `.eq()` filters is like SQL: `DELETE FROM subject_students WHERE student_id = X AND subject_id = Y`.

```
def get_student_subjects(student_id):
    response = supabase.table('subject_students').select(', subjects()').eq('student_id', student_id).execute()
    return response.data
```

Explanation: Gets all subjects a student is enrolled in. `subjects(*)` is a Supabase join -- it automatically fetches the full subject details from the subjects table through the foreign key.

```
def get_student_attendance(student_id):
    response = supabase.table('attendance_logs').select(', subjects()').eq('student_id', student_id).execute()
    return response.data
```

Explanation: Gets all attendance records for a student, with subject details joined. Returns data like: `[[attendance_log_data, subjects: {subject_data}]]`.

```
def create_attendance(logs):
    response = supabase.table('attendance_logs').insert(logs).execute()
    return response.data
```

Explanation: Bulk inserts attendance records. `logs` is a list of dictionaries -- Supabase's `.insert()` can accept multiple records at once, which is much faster than inserting one at a time.

```
def get_attendance_for_teacher(teacher_id):
    response = supabase.table('attendance_logs').select(
        ", subjects!inner(), students(*)"
    ).eq('subjects.teacher_id', teacher_id).execute()
    return response.data
```

Explanation: Fetches all attendance records for a teacher's subjects. Key concepts:

- `subjects!inner(*)` -- an inner join on the subjects table. `!inner` means "only return rows where this join actually matches" (like SQL `INNER JOIN`)
 - `students(*)` -- a regular join to get student names
 - `.eq('subjects.teacher_id', teacher_id)` -- filters on the joined table's column, not the `attendance_logs` table itself
-
-

4. `src/pipelines/face_pipeline.py` -- Face Recognition Engine (119)

lines)

> [Pin] This is the core AI engine for face recognition. It uses dlib's HOG detector, a shape predictor for facial landmarks, and a deep learning model for face embeddings.

```
import dlib
import numpy as np
import face_recognition_models
from sklearn.svm import SVC
import streamlit as st
from src.database.db import get_all_students
```

Explanation: Key imports:

- dlib -- C++ machine learning library with pre-trained face detection/recognition models
- numpy -- numerical computing (arrays, math operations)
- face_recognition_models -- a package that bundles pre-trained model files for dlib
- SVC (Support Vector Classifier) -- a machine learning classifier from scikit-learn
- get_all_students -- to fetch stored face embeddings from the database

[Gear] Model Loading

```
@st.cache_resource
def load_dlib_models():
    detector = dlib.get_frontal_face_detector()
```

Explanation:

- @st.cache_resource -- a Streamlit decorator that caches the function's return value. These models are large and expensive to load. Without caching, they'd reload on every page rerun (every button click!). With caching, they load once and stay in memory.
- dlib.get_frontal_face_detector() -- loads dlib's HOG-based (Histogram of Oriented Gradients) face detector. HOG analyzes gradient patterns in the image to find rectangular regions that look like faces.

```
sp = dlib.shape_predictor(
    face_recognition_models.pose_predictor_model_location()
)
```

Explanation: Loads the shape predictor model, which finds 68 facial landmarks (eyes, nose, mouth corners, jawline, etc.) on a detected face. face_recognition_models.pose_predictor_model_location() returns the file path to the pre-trained model bundled in the package.

```
facerec = dlib.face_recognition_model_v1(
    face_recognition_models.face_recognition_model_location()
)
return detector, sp, facerec
```

Explanation: Loads the face recognition model -- a deep neural network (ResNet) that converts a face image into a 128-dimensional embedding vector. Two photos of the same person will produce similar vectors; different people will have very different vectors. Returns all three models as a tuple.

Face Embedding Extraction

```
def get_face_embeddings(image_np):
    detector, sp, facerec = load_dlib_models()
    faces = detector(image_np, 3)
```

Explanation:

- image_np -- a NumPy array representing the image (pixels as numbers)
- load_dlib_models() -- retrieves models from cache (instant after first load)
- detector(image_np, 3) -- runs face detection. The 3 is the upsample count -- it enlarges the image 3 times before scanning, which helps detect smaller/farther faces but takes longer

```
encodings = []
for face in faces:
    shape = sp(image_np, face)
    face_descriptor = facerec.compute_face_descriptor(image_np, shape, 1)
    encodings.append(np.array(face_descriptor))
return encodings
```

Explanation: For each detected face:

- sp(image_np, face) -- find 68 facial landmarks within the face rectangle
- facerec.compute_face_descriptor(image_np, shape, 1) -- compute the 128-dimensional embedding. The 1 is num_jitters (how many times to re-sample the face -- higher = more accurate but slower)
- Convert to a NumPy array and add to the list

[Key] The returned list of embeddings is the mathematical "fingerprint" of each face in the photo.

[Robot] SVM Classifier Training

```
@st.cache_resource
def get_trained_model():
    X = []
    y = []
    student_db = get_all_students()

    if not student_db:
        return None
```

Explanation: Builds a machine learning classifier from all registered student faces.

- X = list of face embeddings (features)
- y = list of corresponding student IDs (labels)
- @st.cache_resource = model is trained once and cached until explicitly cleared

```
for student in student_db:
    embedding = student.get('face_embedding')
    if embedding:
        X.append(np.array(embedding))
        y.append(student.get('student_id'))

if len(X) == 0:
```

```
return None
```

Explanation: Loops through all students, collecting their face embeddings and IDs. Students without face embeddings are skipped. If no embeddings exist at all, return None (can't train without data).

```
clf = SVC(kernel='linear', probability=True, class_weight='balanced')
try:
    clf.fit(X, y)
except ValueError:
    return None
return {'clf': clf, 'X': X, 'y': y}
```

Explanation: Creates and trains an SVM (Support Vector Machine) classifier:

- kernel='linear' -- uses a linear decision boundary. For face embeddings (which are already highly processed by the neural network), linear separation works well
- probability=True -- enables probability estimates (confidence scores for each prediction)
- class_weight='balanced' -- adjusts weights so students with fewer training photos aren't disadvantaged
- clf.fit(X, y) -- trains the model on the data
- ValueError is caught in case there's only one class (you need at least 2 students to train an SVM)
- Returns a dictionary with the classifier AND the raw training data (used later for distance matching)

[Cycle] Cache Clearing for Retraining

```
def train_classifier():
    """Clear cached model and rebuild it with the latest students from DB."""
    get_trained_model.clear()
    model_data = get_trained_model()
    return bool(model_data)
```

Explanation: When a new student registers, the cached SVM model is outdated. This function:

- .clear() -- clears ONLY this function's cache (not all cached resources)
- Calls get_trained_model() again, which re-fetches students and retrains
- Returns True if training succeeded, False if not

[Search] Attendance Prediction

```
def predict_attendance(class_image_np):
    encodings = get_face_embeddings(class_image_np)
    detected_student = {}

    model_data = get_trained_model()
    if not model_data:
        if student_db_empty := (get_all_students() == []):
            st.error("[Warning] Cannot connect to database...")
        return detected_student, [], len(encodings)
```

Explanation: The main attendance function:

- Extract face embeddings from the classroom photo
- Create empty dictionary for detected students
- Load the trained model (from cache)
- If no model exists, check why -- if the database is empty, show an error. The `:=` is Python's walrus operator (assigns and evaluates in one expression)

```
clf = model_data['clf']
X_train = model_data['X']
y_train = model_data['y']

THRESHOLD = 0.45
```

Explanation: Extracts the classifier and training data. THRESHOLD = 0.45 is the Euclidean distance threshold -- the maximum distance between two face embeddings to consider them the same person. Lower = stricter matching. dlib's default suggestion is 0.6, but 0.45 is used here for higher accuracy (fewer false positives).

```
all_students = sorted(list(set(y_train)))

for encoding in encodings:
    best_match_id = None
    best_match_score = float('inf')

    for stored_id, stored_emb in zip(y_train, X_train):
        dist = np.linalg.norm(stored_emb - encoding)
        if dist < best_match_score:
            best_match_score = dist
            best_match_id = stored_id
```

Explanation: For each detected face in the photo:

- Initialize "best match" variables (worst case: infinite distance)
- Compare against every stored embedding using Euclidean distance (np.linalg.norm calculates the straight-line distance between two 128-dimensional points)
- Track which stored student has the smallest distance (most similar face)

```
if best_match_id is not None and best_match_score <= THRESHOLD:
    detected_student[int(best_match_id)] = True

return detected_student, all_students, len(encodings)
```

Explanation: Only accept the match if the distance is below the threshold (0.45). This prevents random faces from being falsely identified as known students. Returns:

- detected_student -- dictionary of {student_id: True} for everyone recognized
- all_students -- list of all known student IDs
- len(encodings) -- how many faces were detected in the photo

[Bulb] Why not just use the SVM directly? The SVM would always pick the "closest" class, even for a stranger's face. The Euclidean distance check adds a critical safety layer -- it ensures the face is actually close enough to a known student before accepting the match.

5. src/pipelines/voice_pipeline.py -- Voice Recognition Engine (76 lines)

> [Pin] This is the voice-based attendance system. It uses Resemblyzer to create voice embeddings (d-vectors) and cosine similarity for speaker identification.

```
from resemblyzer import VoiceEncoder, preprocess_wav
import numpy as np
import io
import librosa
import streamlit as st
```

Explanation:

- VoiceEncoder -- the deep learning model that converts voice audio into a 256-dimensional d-vector (speaker embedding)
 - preprocess_wav -- normalizes and trims audio for consistent processing
 - librosa -- audio analysis library (loading, resampling, splitting)
 - io -- for reading audio bytes in memory (without saving to disk)
-

Voice Encoder Loading

```
@st.cache_resource
def load_voice_encoder():
    return VoiceEncoder()
```

Explanation: Loads the pre-trained voice encoder model. Just like the face models, @st.cache_resource ensures it only loads once. VoiceEncoder() downloads/loads a deep neural network trained on thousands of speakers.

Voice Embedding Extraction

```
def get_voice_embedding(audio_bytes):
    try:
        encoder = load_voice_encoder()
        audio, sr = librosa.load(io.BytesIO(audio_bytes), sr=16000)
```

Explanation:

- audio_bytes -- raw audio data from the microphone
- librosa.load() -- loads the audio. io.BytesIO(audio_bytes) wraps raw bytes into a file-like object so librosa can read it
- sr=16000 -- resample to 16kHz. Speech recognition models are trained on 16kHz audio, so all input must be at this sample rate

```
        wav = preprocess_wav(audio)
        embedding = encoder.embed_utterance(wav)
        return embedding.tolist()
    except Exception as e:
```

```
st.error('Voice recog error')
return None
```

Explanation:

- preprocess_wav(audio) -- normalizes volume and trims silence from the audio
 - encoder.embed_utterance(wav) -- generates the d-vector (a 256-dimensional float array that uniquely represents this speaker's voice characteristics)
 - .tolist() -- converts NumPy array to a Python list for JSON-serializable storage in Supabase
-

[Search] Speaker Identification

```
def identify_speaker(new_embedding, candidates_dict, threshold=0.65):
    if new_embedding is None or not candidates_dict:
        return None, 0.0

    best_sid = None
    best_score = -1.0
```

Explanation: Identifies who is speaking by comparing a voice embedding against a dictionary of known speakers. threshold=0.65 is the minimum cosine similarity score to accept a match (range: -1 to 1, where 1 = identical).

```
for sid, stored_embedding in candidates_dict.items():
    if stored_embedding:
        similarity = np.dot(new_embedding, stored_embedding)
        if similarity > best_score:
            best_score = similarity
            best_sid = sid
```

Explanation: Loops through all candidate speakers:

- np.dot(new_embedding, stored_embedding) -- computes the dot product of two vectors. Since Resemblyzer's d-vectors are L2-normalized (unit length), the dot product equals the cosine similarity. Higher value = more similar voices.
- Tracks the speaker with the highest similarity score

```
if best_score >= threshold:
    return best_sid, best_score
return None, best_score
```

Explanation: Only returns a match if the similarity meets the threshold (0.65). Below that, the voice is too different -- could be an unknown person or background noise.

Bulk Audio Processing

```
def process_bulk_audio(audio_bytes, candidates_dict, threshold=0.65):
    try:
        encoder = load_voice_encoder()
        audio, sr = librosa.load(io.BytesIO(audio_bytes), sr=16000)
        segments = librosa.effects.split(audio, top_db=30)
```

Explanation: Processes a long classroom audio recording to identify multiple speakers:

- `librosa.effects.split(audio, top_db=30)` -- Voice Activity Detection (VAD). Splits the audio into segments where someone is speaking. `top_db=30` means silence is defined as anything 30dB below the loudest point. Returns a list of (start_sample, end_sample) tuples.

```
identified_results = {}

for start, end in segments:
    if (end - start) < sr * 0.5:
        continue
```

Explanation:

- `(end - start) < sr * 0.5` -- filters out segments shorter than 0.5 seconds. Since `sr=16000` (samples per second), `sr * 0.5 = 8000` samples = 0.5 seconds. Very short segments are usually noise, not speech.

```
segment_audio = audio[start:end]
wav = preprocess_wav(segment_audio)
embedding = encoder.embed_utterance(wav)

sid, score = identify_speaker(embedding, candidates_dict, threshold)

if sid:
    if sid not in identified_results or score > identified_results[sid]:
        identified_results[sid] = score

return identified_results
except Exception as e:
    st.error('Bulk process error')
return {}
```

Explanation: For each valid speech segment:

- Extract the audio slice
- Preprocess and generate an embedding
- Try to identify the speaker against known candidates
- If matched, store the result -- keeping only the highest confidence score per student (a student might speak multiple times; we keep the best match)

Returns a dictionary of {student_id: confidence_score}.

6. src/screens/home_screen.py -- Landing Page (36 lines)

> [Pin] The first screen users see -- choose between Student and Teacher portals.

```
import streamlit as st
from src.components.snapheader import show_header
from src.components.footer import footer_home
from src.ui.base_layout import style_background_home, style_base_layout
```

Explanation: Imports the visual components: the animated hero header, the footer, and the styling functions for the

dark-themed home page.

```
def home_screen():
    show_header()
    style_background_home()
    style_base_layout()
```

Explanation: Sets up the page visuals:

- show_header() -- renders the animated VIBE ID logo with pulse rings and shimmer title
 - style_background_home() -- injects the dark aurora background CSS and particle canvas
 - style_base_layout() -- applies global font imports and button styling
-

```
col1, col2 = st.columns(2, gap="large")
with col1:
    st.header("I 'm Student")
    st.image("https://i.ibb.co/844D9Lrt/mascot-student.png", width=120)
    if st.button('Student Portal', type='primary', icon=':material/arrow_outward:', icon_position='right'):
        st.session_state['login_type'] = 'student'
        st.rerun()
```

Explanation: Creates a two-column layout:

- st.columns(2) -- splits the page into two equal columns
- with col1: -- everything inside goes in the left column
- st.image() -- displays a mascot image from a URL
- Button click sets login_type to 'student' and calls st.rerun(), which restarts the script. On the next run, app.py's match-case routes to student_screen()

```
with col2:
    st.header("I 'm Teacher")
    st.image("https://i.ibb.co/CsmQQV6X/mascot-prof.png", width=145)
    if st.button('Teacher Portal', type='primary', icon=':material/arrow_outward:', icon_position='right'):
        st.session_state['login_type'] = 'teacher'
        st.rerun()

footer_home()
```

Explanation: Same pattern for the teacher column. footer_home() adds the "Created by GHOTRA FIRM" text at the bottom.

7. src/screens/student_screen.py -- Student Login & Dashboard (188 lines)

> [Pin] Handles student face-based login, new student registration (with optional voice), and the student dashboard.

```
import streamlit as st
from src.ui.base_layout import style_background_dashboard, style_base_layout
from src.components.snapheader import header_dashboard
from src.components.footer import footer_dashboard
from PIL import Image
import numpy as np
from src.pipelines.face_pipeline import predict_attendance, get_face_embeddings, train_classifier
from src.pipelines.voice_pipeline import get_voice_embedding
from src.database.db import get_all_students, create_student, get_student_subjects, get_student_attendance, u
import time
from src.components.dialog_enroll import enroll_dialog
from src.components.subject_card import subject_card
```

Explanation: Imports everything needed: UI components, face/voice pipelines, database functions, and the enrollment dialog. PIL.Image is used to open camera captures, and numpy converts images to arrays for the AI pipeline.

[Chart] Student Dashboard

```
def student_dashboard():
    student_data = st.session_state.student_data
    student_id = student_data['student_id']
```

Explanation: Retrieves the logged-in student's data from session state (stored there during login).

```
c1, c2 = st.columns(2, vertical_alignment='center', gap='xxlarge')
with c1:
    header_dashboard()
with c2:
    st.subheader(f""""Welcome, {student_data['name']} """)
    if st.button("Logout", type='secondary', key='loginbackbtn', shortcut="control+backspace", use_contai
        st.session_state['is_logged_in'] = False
        del st.session_state.student_data
        st.rerun()
```

Explanation: Header bar with logo on left, welcome message + logout button on right. Logout clears the session state and reruns. shortcut="control+backspace" adds a keyboard shortcut. del st.session_state.student_data removes the key entirely.

```
c1, c2 = st.columns(2)
with c1:
    st.header('Your Enrolled Subjects')
with c2:
    if st.button('Enroll in Subject', type='primary', use_container_width=True):
        enroll_dialog()
```

Explanation: Section header with an "Enroll" button that opens a dialog for entering a subject code.

```
with st.spinner('Loading your enrolled subjects..'):
    subjects = get_student_subjects(student_id)
    logs = get_student_attendance(student_id)
```

Explanation: `st.spinner()` shows a loading animation while the database queries run. Fetches both enrolled subjects and attendance records.

```
stats_map = {}
for log in logs:
    sid = log['subject_id']
    if sid not in stats_map:
        stats_map[sid] = {"total": 0, "attended": 0}
    stats_map[sid]['total'] += 1
    if log.get('is_present'):
        stats_map[sid]['attended'] += 1
```

Explanation: Builds attendance statistics per subject by looping through all attendance logs:

- total = total attendance sessions recorded
- attended = sessions where `is_present` was True
- Groups stats by `subject_id` in a dictionary

```
cols = st.columns(2)
for i, sub_node in enumerate(subjects):
    sub = sub_node['subjects']
    sid = sub['subject_id']
    stats = stats_map.get(sid, {"total": 0, "attended": 0})
```

Explanation: Creates a 2-column grid for subject cards. `sub_node['subjects']` unwraps the Supabase join -- the response structure is `{enrollment_data, subjects: {subject_data}}`.

```
with cols[i % 2]:
    subject_card(
        name=sub['name'],
        code=sub['subject_code'],
        section=sub['section'],
        stats=[
            ('[Calendar]', 'Total', stats['total']),
            ('[Check]', 'Attended', stats['attended']),
        ],
        footer_callback=lambda: unenroll_button(sub['subject_id'])
    )
```

Explanation: `cols[i % 2]` alternates between left (0) and right (1) columns -- this creates a grid layout. Each card shows the subject name, code, section, and attendance stats.

Student Login (Face Recognition)

```
def student_screen():
    style_background_dashboard()
    style_base_layout()

    if "student_data" in st.session_state:
        student_dashboard()
    return
```

Explanation: Entry point for the student screen. If the student is already logged in (their data exists in session state), skip straight to the dashboard.

```
st.header('Login using FaceID', text_alignment='center')
photo_source = st.camera_input("Position your face in the center")
```

Explanation: `st.camera_input()` opens the device camera and lets the user take a photo. Returns a file-like object when a photo is captured, or `None` if no photo taken yet.

```
if photo_source:
    img = np.array(Image.open(photo_source))
    with st.spinner('AI is scanning..'):
        detected, all_ids, num_faces = predict_attendance(img)
```

Explanation: When a photo is taken:

- Open it with PIL and convert to a NumPy array (the format dlib expects)
- Run the face prediction pipeline

```
if num_faces == 0:
    st.warning('Face not found!')
elif num_faces > 1:
    st.warning('Multiple faces found')
else:
    if detected:
        student_id = list(detected.keys())[0]
        all_students = get_all_students()
        student = next((s for s in all_students if s['student_id'] == student_id), None)
        if student:
            st.session_state.is_logged_in = True
            st.session_state.user_role = 'student'
            st.session_state.student_data = student
            st.toast(f"Welcome Back {student['name']}")
            time.sleep(1)
            st.rerun()
```

Explanation: Login logic:

- 0 faces -> warning
- Multiple faces -> warning (login requires exactly one face)
- 1 face detected -> check if it matches a known student
- `list(detected.keys())[0]` -- get the first (only) matched student ID
- `next((s for s in all_students if ...), None)` -- generator expression to find the full student record
- If found, store everything in session state and rerun (which triggers the dashboard)
- `time.sleep(1)` -- brief pause so the user can see the welcome toast

```
else:
    st.info('Face not recognized! You might be a new student!')
    show_registration = True
```

Explanation: If no match found, the face is unknown -- offer registration.

[Memo] Student Registration

```
if show_registration:
    with st.container(border=True):
        st.header('Register new Profile')
        new_name = st.text_input("Enter your name", placeholder='E.g. Hamza Rizvi')

        st.subheader('Optional : Voice Enrollment')
        audio_data = None
        try:
            audio_data = st.audio_input('Record a short phrase...')
        except Exception:
            st.error('Audio Data failed!')
```

Explanation: Registration form with:

- Name text input
- Optional voice recording using `st.audio_input()` (wrapped in try/except because some browsers don't support microphone access)

```
if st.button('Create Account', type='primary'):
    if new_name:
        with st.spinner('Creating profile..'):
            img = np.array(Image.open(photo_source))
            encodings = get_face_embeddings(img)
            if encodings:
                face_emb = encodings[0].tolist()

                voice_emb = None
                if audio_data:
                    voice_emb = get_voice_embedding(audio_data.read())

                response_data = create_student(new_name, face_embedding=face_emb, voice_embedding=voice_emb)

            if response_data:
                train_classifier()
                st.session_state.is_logged_in = True
                st.session_state.user_role = 'student'
                st.session_state.student_data = response_data[0]
                st.toast(f'Profile Created! Hi {new_name}!')
                time.sleep(1)
                st.rerun()
```

Explanation: Registration flow:

- Extract face embedding from the photo (`.tolist()` converts NumPy array to Python list for JSON storage)
- If voice audio was recorded, generate a voice embedding too
- Create the student record in the database
- `train_classifier()` -- crucial step! Clears the cached SVM model and retrains it with the new student's data
- Log the student in immediately after registration

8. src/screens/teacher_screen.py -- Teacher Dashboard (475 lines)

> [Pin] The most complex screen -- handles login, registration, subject management, and attendance taking (both face and voice).

Routing Logic

```
def teacher_screen():
    style_background_dashboard()
    style_base_layout()

    if "teacher_data" in st.session_state:
        teacher_dashboard()
    elif 'teacher_login_type' not in st.session_state or st.session_state.teacher_login_type == "login":
        teacher_screen_login()
    elif st.session_state.teacher_login_type == "register":
        teacher_screen_register()
```

Explanation: Three-way routing for the teacher section:

- If teacher_data exists -> already logged in -> show dashboard
- If teacher_login_type is "login" (or not set) -> show login form
- If teacher_login_type is "register" -> show registration form

Teacher Login

```
def teacher_screen_login():
    # ... header and back button setup ...

    teacher_username = st.text_input("Enter username", placeholder='ananyaroy')
    teacher_pass = st.text_input("Enter password", type='password', placeholder="Enter password")
```

Explanation: type='password' makes the input field show dots instead of characters (standard password field behavior).

```
with btnc1:
    if st.button('Login', icon=':material/passkey:', shortcut='control+enter', use_container_width=True):
        if login_teacher(teacher_username, teacher_pass):
            st.toast("welcome back!", icon="")
            import time
            time.sleep(1)
            st.rerun()
        else:
            st.error("Invalid username and password combo")
```

Explanation: Login button calls login_teacher() helper. shortcut='control+enter' lets users press Ctrl+Enter instead of clicking. st.toast() shows a temporary notification popup.

```
def login_teacher(username, password):
    if not username or not password:
        return False
```

```

teacher = teacher_login(username, password)
if teacher:
    st.session_state.user_role = 'teacher'
    st.session_state.teacher_data = teacher
    st.session_state.is_logged_in = True
    return True
return False

```

Explanation: Helper function that calls the database `teacher_login()`, and if successful, stores the teacher data in session state.

[Memo] Teacher Registration

```

def register_teacher(teacher_username, teacher_name, teacher_pass, teacher_pass_confirm):
    if not teacher_username or not teacher_name or not teacher_pass:
        return False, "All Fields are required!"
    if check_teacher_exists(teacher_username):
        return False, "Username already taken"
    if teacher_pass != teacher_pass_confirm:
        return False, "Password doesn't match"
    try:
        create_teacher(teacher_username, teacher_pass, teacher_name)
        return True, "Sucessfully Created! Login Now"
    except Exception as e:
        return False, "Unexpected Error!"

```

Explanation: Validation chain before creating a teacher:

- Check all fields are filled
- Check username isn't already taken
- Check passwords match
- Try to create the account

Returns a tuple of (success_bool, message_string) for clean error handling.

[Chart] Teacher Dashboard

```

def teacher_dashboard():
    teacher_data = st.session_state.teacher_data
    # ... header with logout ...

    if "current_teacher_tab" not in st.session_state:
        st.session_state.current_teacher_tab = 'take_attendance'

```

Explanation: Initializes the active tab in session state. Default tab is "Take Attendance".

```

tab1, tab2, tab3 = st.columns(3)
with tab1:
    type1 = "primary" if st.session_state.current_teacher_tab == 'take_attendance' else "tertiary"
    if st.button('Take Attendance', type=type1, use_container_width=True, icon=':material/ar_on_you:'):
        st.session_state.current_teacher_tab = 'take_attendance'

```

```
st.rerun()
```

Explanation: Custom tab navigation built with buttons. The active tab gets type="primary" styling (highlighted), inactive tabs get type="tertiary" (muted). Clicking a tab updates session state and reruns to show the correct content.

```
if st.session_state.current_teacher_tab == "take_attendance":
    teacher_tab_take_attendance()
if st.session_state.current_teacher_tab == "manage_subjects":
    teacher_tab_manage_subjects()
if st.session_state.current_teacher_tab == "attendance_records":
    teacher_tab_attendance_records()
```

Explanation: Renders whichever tab content matches the current selection.

[Camera] Take Attendance Tab

```
def teacher_tab_take_attendance():
    teacher_id = st.session_state.teacher_data['teacher_id']

    if 'attendance_images' not in st.session_state:
        st.session_state.attendance_images = []

    subjects = get_teacher_subjects(teacher_id)
    if not subjects:
        st.warning('You havent created any subjects yet!')
    return
```

Explanation: Initializes the image list in session state (persists across reruns) and fetches the teacher's subjects.

```
subject_options = {f"{s['name']} - {s['subject_code']}": s['subject_id'] for s in subjects}
selected_subject_label = st.selectbox('Select Subject', options=list(subject_options.keys()))
selected_subject_id = subject_options[selected_subject_label]
```

Explanation: Dictionary comprehension creates a mapping of "Subject Name - CODE" -> subject_id. The selectbox shows human-readable labels, but we store the ID for database operations.

```
if st.session_state.attendance_images:
    gallery_cols = st.columns(4)
    for idx, img in enumerate(st.session_state.attendance_images):
        with gallery_cols[idx % 4]:
            st.image(img, width='stretch', caption=f'Photo {idx+1}')
```

Explanation: Displays uploaded photos in a 4-column grid. idx % 4 distributes images across columns evenly.

[Brain] Face Analysis Flow

```
if st.button('Run Face Analysis', ...):
    with st.spinner('Deep scanning classroom photos...'):
        all_detected_ids = {}

        for idx, img in enumerate(st.session_state.attendance_images):
            img_np = np.array(img.convert('RGB'))
```

```

detected, _, _ = predict_attendance(img_np)

if detected:
    for sid in detected.keys():
        student_id = int(sid)
        all_detected_ids.setdefault(student_id, []).append(f"Photo {idx+1}")

```

Explanation: For each uploaded photo:

- Convert to RGB NumPy array (.convert('RGB') ensures consistent color format)
- Run face prediction
- .setdefault(student_id, []).append(...) -- if this student wasn't seen before, create an empty list, then append which photo they were found in. This tracks which photos each student appeared in.

```

enrolled_res = supabase.table('subject_students').select(", students()").eq('subject_id', selected_subject_id)
enrolled_students = enrolled_res.data

```

Explanation: Fetches all students enrolled in the selected subject (with their details via join). This is needed to build the full attendance report (present + absent).

```

results, attendance_to_log = [], []
current_timestamp = datetime.now().strftime("%Y-%m-%dT%H:%M:%S")

for node in enrolled_students:
    student = node['students']
    sources = all_detected_ids.get(int(student['student_id']), [])
    is_present = len(sources) > 0

    results.append({
        "Name": student['name'],
        "ID": student['student_id'],
        "Source": ", ".join(sources) if is_present else "-",
        "Status": "[Check] Present" if is_present else "[X] Absent"
    })

    attendance_to_log.append({
        'student_id': student['student_id'],
        'subject_id': selected_subject_id,
        'timestamp': current_timestamp,
        'is_present': bool(is_present)
    })

```

Explanation: For every enrolled student:

- Check if they were detected in any photo
- Build a display result (for the UI table) and a database log entry
- All logs share the same timestamp -- this groups them as one "session"

```

attendance_result_dialog(pd.DataFrame(results), attendance_to_log)

```

Explanation: Opens the attendance results dialog with a pandas DataFrame for display and the raw logs for database saving.

[Clipboard] Manage Subjects Tab

```
def teacher_tab_manage_subjects():
    teacher_id = st.session_state.teacher_data['teacher_id']
    # ... header and "Create New Subject" button ...

    subjects = get_teacher_subjects(teacher_id)
    if subjects:
        for sub in subjects:
            stats = [
                ("", "Students", sub['total_students']),
                ("[Clock]", "Classes", sub['total_classes']),
            ]
            # ... share button + subject_card rendering ...
```

Explanation: Lists all subjects with student count and class count stats. Each card has a "Share" button that opens a dialog with a QR code.

[Chart] Attendance Records Tab

```
def teacher_tab_attendance_records():
    teacher_id = st.session_state.teacher_data['teacher_id']
    records = get_attendance_for_teacher(teacher_id)
```

Explanation: Fetches all attendance records across all of this teacher's subjects.

```
data = []
for r in records:
    ts = r.get('timestamp')
    data.append({
        "ts_group": ts.split(".")[0] if ts else None,
        "Time": datetime.fromisoformat(ts).strftime("%Y-%m-%d %I:%M %p") if ts else "N/A",
        "Subject": r['subjects']['name'],
        "Subject Code": r['subjects']['subject_code'],
        "Student Name": r['students']['name'] if r.get('students') else "Unknown",
        "Student ID": r['students']['student_id'] if r.get('students') else "-",
        "is_present": bool(r.get('is_present', False))
    })
df = pd.DataFrame(data)
```

Explanation: Transforms raw database records into a pandas DataFrame. `ts.split(".")[0]` removes microseconds from the timestamp for clean grouping. `datetime.fromisoformat().strftime()` converts ISO format to human-readable "2025-06-19 08:30 PM".

```
summary = (
    df.groupby(['ts_group', 'Time', 'Subject', 'Subject Code'])
    .agg(
        Present_Count=('is_present', 'sum'),
        Total_Count=('is_present', 'count')
    ).reset_index()
)

summary['Attendance Stats'] = (
    "[Check] " + summary['Present_Count'].astype(str) + " / "
    + summary['Total_Count'].astype(str) + " Students"
)
```

Explanation: Powerful pandas aggregation:

- `.groupby()` -- groups all records by session timestamp + subject
- `.agg()` -- sum of `is_present` = count of True values (present students); count = total students
- Creates a formatted string like "[Check] 25 / 30 Students"

```
selection_event = st.dataframe(
    display_df,
    width='stretch',
    hide_index=True,
    on_select="rerun",
    selection_mode="single-row"
)

if selection_event and selection_event.selection.rows:
    selected_row_idx = selection_event.selection.rows[0]
    # ... drill-down into selected row ...
```

Explanation: `on_select="rerun"` makes the dataframe interactive -- clicking a row triggers a rerun. `selection_mode="single-row"` allows selecting exactly one row. When a row is clicked, it shows a detailed breakdown of which students were present/absent in that session.



9. src/components/ -- All Dialog & UI Components

[Pin] snapheader.py -- Hero Header with CSS Animations (181 lines)

```
def show_header():
    logo_url = "https://i.ibb.co/YTYGn5qV/logo.png"
    st.markdown(f"""
<style>
@keyframes pulsRing {{
    0%   {{ transform: scale(1);   opacity: 0.7; }}
    70%  {{ transform: scale(1.7); opacity: 0;   }}
    100% {{ transform: scale(1.7); opacity: 0;   }}
}}
</style>
""", unsafe_allow_html=True)
```

Explanation: The hero header for the home page. It uses CSS keyframe animations injected via `st.markdown()` with `unsafe_allow_html=True`:

- `pulsRing` -- creates expanding rings behind the logo that fade out (like a radar pulse)
- `shimmerTitle` -- moves a gradient across the title text creating a shimmering effect
- `fadeUp` -- elements slide up and fade in when the page loads
- `badgePop` -- feature badges pop in with a scale animation

[Key] The `{{ }}` double braces are needed because f-strings use `{}` -- doubling them escapes them for literal CSS braces.

The HTML structure creates:

- A logo with 3 animated pulse rings behind it (staggered with animation-delay)
- The "VIBE ID" title with a gradient shimmer effect
- Feature badges ("[Brain] Face Recognition", "[Lightning] AI Powered", "[Camera] Instant Attendance")

```
def header_dashboard():
    logo_url = "https://i.ibb.co/YTYGn5qV/logo.png"
    st.markdown(f"""
<div style="display:flex; align-items:center; justify-content:flex-start; gap:15px; margin-bottom:20px;">
    <img src='{logo_url}' style='height:60px; filter: drop-shadow(0px 0px 4px #228BE6);' />
    <h2 style='text-align:left; color:#5865F2; font-family:sans-serif; margin:0;'>VIBE ID</h2>
</div>
""", unsafe_allow_html=True)
```

Explanation: A smaller, simpler header used on the dashboard pages. Shows the logo + "VIBE ID" text in a horizontal flex layout with a blue drop shadow on the logo.

[Pin] footer.py -- Footer Components (24 lines)

```
def footer_home():
    st.markdown(f"""
    <div style="margin-top:2rem; display:flex; gap:6px; justify-content:center; items-align:center">
    <p style="font-weight:bold; color:white;"> Created by GHOTRA FIRM </p>
    </div>
    """, unsafe_allow_html=True)

def footer_dashboard():
    # Same but with color:black for the light dashboard theme
```

Explanation: Two footer variants -- footer_home() has white text (for the dark home page background) and footer_dashboard() has black text (for the light dashboard background).

[Pin] subject_card.py -- Reusable Card Component (23 lines)

```
def subject_card(name, code, section, stats=None, footer_callback=None):
    html = f"""
    <div style="background:white; border-left: 8px solid #4f46e5; padding:25px;
        border-radius: 20px; border: 1px solid rgba(15,23,42,0.08);
        box-shadow: 0 4px 20px rgba(0,0,0,0.03); margin-bottom:20px;">
    <h3 style="...">{name}</h3>
    <p style="...">Code: <span style="...">{code}</span> | Section: {section}</p>
    """
```

Explanation: A reusable HTML card component with:

- White background with a thick purple left border (accent stripe)
- Rounded corners and subtle shadow
- Subject name as heading, code in a pill/badge style, and section

```
if stats:
    html += """<div style="display:flex; gap:8px; flex-wrap:wrap; margin-bottom: 5px;">"""
```

```

    for icon, label, value in stats:
        html += f'<div style="...">{icon} <b>{value}</b> {label}</div>'
    html += "</div>"

st.markdown(html, unsafe_allow_html=True)

if footer_callback:
    footer_callback()

```

Explanation: The stats parameter accepts a list of tuples like `[('Calendar', 'Total', 5), ('Check', 'Attended', 3)]` and renders them as small stat badges. The footer_callback is a function that gets called after the HTML -- this is how buttons (like "Unenroll" or "Share") are added below the card. It uses the callback pattern for flexibility.

[Pin] dialog_add_photo.py -- Camera/Upload Photos (50 lines)

```

@st.dialog("Capture or upload photos")
def add_photos_dialog():
    if 'photo_tab' not in st.session_state:
        st.session_state.photo_tab = 'camera'

```

Explanation: `@st.dialog("title")` is a Streamlit decorator that wraps the function in a modal dialog (popup overlay). Everything inside runs in the dialog context. Tracks which tab (camera/upload) is active via session state.

```

if st.session_state.photo_tab == 'camera':
    cam_photo = st.camera_input('Take Snapshot', key='dialog_cam')
    if cam_photo:
        st.session_state.attendance_images.append(Image.open(cam_photo))
        st.toast('Photo Captured')
        st.rerun()

```

Explanation: Camera tab -- captures a photo and appends it to the shared attendance_images list in session state. `Image.open()` converts the raw camera data to a PIL Image object.

```

if st.session_state.photo_tab == 'upload':
    uploaded_files = st.file_uploader('choose image files', type=['jpg', 'png', 'jpeg'],
                                      accept_multiple_files=True, key='dialog_upload')

    if uploaded_files:
        for f in uploaded_files:
            st.session_state.attendance_images.append(Image.open(f))
            st.toast('Photo Uploaded Successfully')
            st.rerun()

```

Explanation: Upload tab -- allows selecting multiple image files at once. `accept_multiple_files=True` enables multi-select.

[Pin] dialog_attendance_results.py -- Review & Confirm Attendance (73 lines)

```

def show_attendance_result(df, logs):
    if 'Source' in df.columns:
        df['Source'] = df['Source'].fillna('-').astype(str)

```

Explanation: Sanitizes the DataFrame -- `fillna('-')` replaces any NaN values with "-" and `.astype(str)` forces the column to

string type. This prevents Streamlit's dataframe renderer from crashing on mixed types.

```
st.dataframe(df, hide_index=True, width='stretch')

with col2:
    if st.button('Confirm & Save', use_container_width=True, type='primary'):
        try:
            sanitized_logs = []
            for log in logs:
                sanitized_logs.append({
                    'student_id': int(log['student_id']),
                    'subject_id': str(log['subject_id']),
                    'timestamp': str(log['timestamp']),
                    'is_present': bool(log['is_present'])
                })
            create_attendance(sanitized_logs)
```

Explanation: Before saving to the database, each log entry is sanitized -- values are explicitly cast to the correct types (int, str, bool). This prevents Supabase from rejecting data due to type mismatches (e.g., NumPy int64 instead of Python int).

```
@st.dialog("Attendance Reports")
def attendance_result_dialog(df, logs):
    show_attendance_result(df, logs)
```

Explanation: Thin wrapper -- the dialog decorator creates the popup, and the actual logic is in show_attendance_result() (which is also reused by the voice attendance dialog).

[Pin] dialog_auto_enroll.py -- QR Code Auto Enrollment (43 lines)

```
@st.dialog("Quick Enrollment")
def auto_enroll_dialog(subject_code):
    student_id = st.session_state.student_data['student_id']

    res = supabase.table('subjects').select('subject_id, name').eq('subject_code', subject_code).execute()
    if not res.data:
        st.error('Subject Code not found!')
        return
```

Explanation: Triggered when a student opens a QR code link. First, looks up the subject code to verify it exists.

```
check = supabase.table('subject_students').select('*').eq('subject_id', subject['subject_id']).eq('student_id', student_id)
if check.data:
    st.info('Youre already enrolled!')
    return
```

Explanation: Checks if the student is already enrolled (prevents duplicate enrollment records).

```
st.markdown(f'Would you like to enroll in {subject["name"]}??')
# ... Yes/No buttons ...
if st.button('Yes enroll now!', type='primary', ...):
    enroll_student_to_subject(student_id, subject['subject_id'])
    st.success('Joined successfully!')
    st.query_params.clear()
```

```
time.sleep(2)
st.rerun()
```

Explanation: Shows a confirmation dialog. On "Yes", enrolls the student and clears the URL query parameters (removes ?join-code=CS101 from the URL so the dialog doesn't reopen).

[Pin] dialog_create_subject.py -- Subject Creation Form (23 lines)

```
@st.dialog("Create New Subject")
def create_subject_dialog(teacher_id):
    sub_id = st.text_input("Subject Code", placeholder="CS101")
    sub_name = st.text_input("Subject Name", placeholder="Introduction to Computer Science")
    sub_section = st.text_input("Section", placeholder="A")

    if st.button("Create Subject Now", type='primary', use_container_width=True):
        if sub_id and sub_name and sub_section:
            create_subject(sub_id, sub_name, sub_section, teacher_id)
            st.toast("Subject Created Successfully!")
            st.rerun()
        else:
            st.warning("Please fill all the fields")
```

Explanation: Simple form dialog -- three text inputs and a submit button. Validation ensures all fields are filled. On success, st.rerun() closes the dialog and refreshes the subject list.

[Pin] dialog_enroll.py -- Manual Enrollment (29 lines)

```
@st.dialog("Enroll in Subject")
def enroll_dialog():
    join_code = st.text_input('Subject Code', placeholder='Eg. CS101')

    if st.button('Enroll now', type='primary', use_container_width=True):
        if join_code:
            res = supabase.table('subjects').select('subject_id, name, subject_code').eq('subject_code', join_code)
            if res.data:
                subject = res.data[0]
                student_id = st.session_state.student_data['student_id']
                check = supabase.table('subject_students').select('*').eq('subject_id', subject['subject_id'])
                if check.data:
                    st.warning('You are already enrolled in this program')
                else:
                    enroll_student_to_subject(student_id, subject['subject_id'])
                    st.success('Successfully enrolled!')
```

Explanation: Manual enrollment flow (student types a code instead of scanning QR). Same logic as auto-enroll but without the URL parameter handling. Validates: code exists -> not already enrolled -> enroll.

[Pin] dialog_share_subject.py -- QR Code Generation (32 lines)

```
@st.dialog("Share Class Link")
def share_subject_dialog(subject_name, subject_code):
    app_domain = "snapclass-main.streamlit.app"
    join_url = f"{app_domain}?join-code={subject_code}"

    qr = segno.make(join_url)
    out = io.BytesIO()
    qr.save(out, kind='png', scale=10, border=1)
```

Explanation: Generates a QR code for class enrollment:

- segno.make(join_url) -- creates a QR code object from the URL
- io.BytesIO() -- an in-memory buffer (no need to save to disk)
- .save(out, kind='png', scale=10, border=1) -- renders the QR as a PNG at 10x scale with 1-module border

```
with col1:
    st.code(join_url, language="text")
    st.code(subject_code, language="text")
with col2:
    st.image(out.getvalue(), caption='QRCODE for class joining')
```

Explanation: Shows the copyable link on the left and the QR code image on the right. st.code() renders text in a monospace box with a copy button.

[Pin] dialog_voice_attendance.py -- Voice Attendance (70 lines)

```
@st.dialog('Voice Attendance')
def voice_attendance_dialog(selected_subject_id):
    audio_data = st.audio_input("Record classroom audio")

    if st.button('Analyze Audio', use_container_width=True, type='primary'):
        with st.spinner('Processing Audio data'):
            enrolled_res = supabase.table('subject_students').select(", students()").eq('subject_id', selected_subject_id)
```

Explanation: Audio recording dialog. When "Analyze" is clicked, it fetches all enrolled students to build the candidate list.

```
candidates_dict = {
    s['students']['student_id']: s['students']['voice_embedding']
    for s in enrolled_students if s['students'].get('voice_embedding')
}
```

Explanation: Dictionary comprehension that creates {student_id: voice_embedding} for all students who have registered a voice profile. Students without voice embeddings are excluded.

```
audio_bytes = audio_data.read()
detected_scores = process_bulk_audio(audio_bytes, candidates_dict)
```

Explanation: Sends the raw audio and candidate dict to the voice pipeline, which returns {student_id: confidence_score} for each recognized speaker.

```
for node in enrolled_students:
    student = node['students']
    score = detected_scores.get(student['student_id'], 0.0)
    is_present = bool(score > 0)
```

```
# ... build results and logs ...

st.session_state.voice_attendance_results = (pd.DataFrame(results), attendance_to_log)
```

Explanation: Maps detection results to a full attendance report (same format as face attendance). Results are stored in session state so they persist after the spinner finishes.

```
if st.session_state.get('voice_attendance_results'):
    df_results, logs = st.session_state.voice_attendance_results
    show_attendance_result(df_results, logs)
```

Explanation: If results exist, renders the review/confirm UI (reusing the same show_attendance_result function from the face attendance flow).



10. src/ui/base_layout.py -- UI Themes & Animations (346 lines)

> [Pin] This file contains all CSS styling and the JavaScript particle system that creates the visual identity of the app.

Home Page Background

```
def style_background_home():
    st.markdown("""
<style>
.stApp {
    background: radial-gradient(circle at center, #0a0f1d 0%, #030408 100%) !important;
    overflow: hidden;
}
</style>
""", unsafe_allow_html=True)
```

Explanation: Sets a dark radial gradient background -- deep navy blue (#0a0f1d) in the center fading to near-black (#030408) at the edges. !important overrides Streamlit's default white background.

```
.aurora-orb {
    position: fixed;
    border-radius: 50%;
    filter: blur(90px);
    pointer-events: none;
    z-index: 0;
    animation: driftOrb ease-in-out infinite alternate;
}

.ao1 { width:500px; height:500px; background:rgba(99, 102, 241, 0.12); top:-15%; left:-10%; }
.ao2 { width:380px; height:380px; background:rgba(59, 130, 246, 0.10); bottom:-10%; right:-5%; }
.ao3 { width:280px; height:280px; background:rgba(168, 85, 247, 0.08); top:40%; left:55%; }
.ao4 { width:220px; height:220px; background:rgba(56, 189, 248, 0.06); top:60%; left:10%; }
```

Explanation: Aurora orbs -- four large, blurred circles positioned around the page that slowly drift back and forth. They create a soft, colorful glow effect similar to the Northern Lights:

- filter: blur(90px) -- heavy blur makes them look like soft glowing clouds
- pointer-events: none -- clicks pass through them

- z-index: 0 -- behind all content
- Each orb has a different color (indigo, blue, purple, cyan), size, position, and animation timing

[Sparkles] Particle Canvas (JavaScript)

```
components.html(""...", height=0, scrolling=False)
```

Explanation: components.html() injects raw HTML/JavaScript into the page. height=0 makes the iframe invisible -- the canvas is drawn on the parent page, not inside the iframe.

Key Parts of the Particle System:

```
var cv = pd.createElement('canvas');
cv.style.cssText = 'position:fixed;top:0;left:0;width:100vw;height:100vh:pointer-events:none;z-index:1;';
pb.appendChild(cv);
```

Explanation: Creates a full-screen canvas element on the parent document (not the iframe). pointer-events:none ensures it doesn't block clicks on buttons.

```
var PAL = ['#ffffff', '#e0f2fe', '#bae6fd', '#7dd3fc', '#c7d2fe', '#e0e7ff', '#a5b4fc', '#f0f9ff'];
```

Explanation: Color palette for particles -- all light, icy blue/white tones to complement the dark background.

```
function Particle(x, y, fc) {
  this.fc = !!fc; // "fc" = follower/cursor particle
  this.x = x != null ? x : Math.random() * cv.width;
  this.y = y != null ? y : Math.random() * cv.height;
  this.ox = this.x; this.oy = this.y; // original position (for returning)
  this.vx = (Math.random()-0.5)*0.8; this.vy = (Math.random()-0.5)*0.8;
  this.r = fc ? Math.random()*2+1.5 : Math.random()*1.5+0.8;
  this.life = fc ? Math.random()*50+30 : Infinity;
}
```

Explanation: Two types of particles:

- Ambient particles (fc=false) -- 80 permanent particles that float randomly and return to their original positions. They live forever (Infinity).
- Cursor particles (fc=true) -- spawned at the mouse position, larger (r), and temporary (life = 30-80 frames). They create a trail effect.

```
Particle.prototype.update = function() {
  if (active) {
    var dx = mouse.x-this.x, dy = mouse.y-this.y, d = Math.sqrt(dx*dx+dy*dy);
    if (d < 250) {
      var force = (250-d)/250;
      this.vx += (dx/d)*force*0.18;
      this.vy += (dy/d)*force*0.18;
      if (d < 50) {
        this.vx -= (dx/d)*0.5;
        this.vy -= (dy/d)*0.5;
      }
    }
  }
}
```

```
this.vx = 0.94; this.vy = 0.94; // drag
};
```

Explanation: Mouse interaction physics:

- Particles within 250px are attracted toward the cursor (force increases as they get closer)
- But particles within 50px are repelled -- this creates an orbit effect where particles swirl around the cursor instead of clumping on it
- $\times 0.94$ applies drag (friction) so particles slow down naturally

```
function edges() {
  for (var i=0; i<am.length; i++) {
    for (var j=i+1; j<am.length; j++) {
      var d = Math.sqrt(dx*dx+dy*dy);
      if (d < 110) {
        ctx.strokeStyle = 'rgba(224,242,254,0.35)';
        ctx.beginPath(); ctx.moveTo(a.x,a.y); ctx.lineTo(b.x,b.y); ctx.stroke();
      }
    }
  }
}
```

Explanation: Draws faint connecting lines between ambient particles that are within 110px of each other. This creates a constellation/network effect. Opacity fades with distance ($t = 1 - d/110$).

```
function ring() {
  if (!active) return;
  var rs = [{r:22,w:1.2,a:0.4}, {r:38,w:0.8,a:0.2}, {r:58,w:0.4,a:0.08}];
  rs.forEach(function(r) {
    ctx.arc(mouse.x, mouse.y, r.r, 0, Math.PI*2);
    ctx.stroke();
  });
}
```

Explanation: Draws 3 concentric rings around the mouse cursor in cyan (#38bdf8) with decreasing opacity. Creates a subtle "targeting reticle" effect.

Dashboard Background

```
def style_background_dashboard():
  st.markdown("""
<style>
.stApp {
  background: #f4f6fa !important;
}
.stApp h1, .stApp h2, .stApp h3, .stApp h4 {
  color: #0f172a !important;
}
</style>
""", unsafe_allow_html=True)
```

Explanation: Light theme for the dashboard -- soft gray background with dark navy text. Also resets column styles to prevent the home page's glassmorphism cards from appearing here.

[Art] Base Layout & Typography

```
def style_base_layout():
    st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Climate+Crisis:YEAR@1979&display=swap');
@import url('https://fonts.googleapis.com/css2?family=Outfit:wght@100..900&display=swap');
</style>
""", unsafe_allow_html=True)
```

Explanation: Imports two Google Fonts:

- Climate Crisis -- a bold, decorative display font used for headings (h1, h2)
- Outfit -- a clean, modern sans-serif font used for body text, badges, and labels

```
MainMenu, footer, header { visibility: hidden; }
```

Explanation: Hides Streamlit's default UI chrome (hamburger menu, "Made with Streamlit" footer, and header bar) for a cleaner look.

Button Styling Hierarchy:

```
/ Primary Buttons /
button, button[kind="primary"] {
    border-radius: 1.5rem !important;
    background: linear-gradient(135deg, #4f46e5 0%, #3b82f6 100%) !important;
    box-shadow: 0 4px 12px rgba(79, 70, 229, 0.25) !important;
}

/ Secondary Buttons /
button[kind="secondary"] {
    border: 1px solid rgba(15, 23, 42, 0.12) !important;
}

/ Tertiary Buttons /
button[kind="tertiary"] {
    background-color: rgba(239, 68, 68, 0.1) !important;
    color: #f87171 !important;
}
```

Explanation: Three button tiers:

- Primary -- bold indigo-to-blue gradient with shadow. Main action buttons (Submit, Create, Analyze)
- Secondary -- subtle border, transparent background. Navigation buttons (Go back, Logout)
- Tertiary -- light red tint. Destructive/minor actions (Unenroll, Delete)

All buttons get hover effects: translateY(-2px) scale(1.02) creates a subtle "lift" animation.



[Book] Glossary -- Key Terms Explained

Session State

Streamlit's built-in dictionary (`st.session_state`) that persists data across page reruns. Every user interaction causes Streamlit to re-execute the entire script. Without session state, all variables would reset. Think of it as the app's short-term memory for each user session.

@st.cache_resource

A Streamlit decorator that caches the return value of a function across all users and reruns. Used for expensive objects like ML models or database connections that should load once and stay in memory. The cached value is only cleared manually (`.clear()`) or when the app restarts.

Embedding

A fixed-size numerical vector (list of numbers) that represents complex data in a way computers can compare. Face embeddings are 128-dimensional vectors; voice embeddings are 256-dimensional. Two embeddings from the same person will be "close" in vector space; different people will be "far apart."

SVM (Support Vector Machine)

A machine learning algorithm that finds the best boundary (hyperplane) to separate different classes of data. In VIBE ID, it separates face embeddings of different students. `kernel='linear'` means it uses a flat boundary (works well for pre-processed embeddings). `class_weight='balanced'` adjusts for uneven training data sizes.

Cosine Similarity

A measure of how similar two vectors are based on the angle between them (ignoring magnitude). Range: -1 (opposite) to 1 (identical). Used in voice recognition because it's robust to volume differences. Formula: $\cos() = (A \cdot B) / (|A| \times |B|)$. When vectors are normalized (unit length), it simplifies to just the dot product.

Euclidean Distance

The "straight-line" distance between two points in multi-dimensional space. Formula: $((a - b)^2)$. Used in face recognition -- lower distance = more similar faces. A threshold of 0.45 means "if two face embeddings are less than 0.45 apart, they're probably the same person."

bcrypt

A one-way password hashing algorithm designed to be intentionally slow (to resist brute-force attacks). It incorporates a random salt (extra random data) into each hash, so even identical passwords produce different hashes. Used to securely store teacher passwords.

d-vector

A speaker embedding produced by Resemblyzer's deep neural network. "d" stands for "deep" (deep learning). It captures the unique voice characteristics of a speaker (pitch, timbre, speaking style) as a 256-dimensional vector. Two

recordings of the same person will produce similar d-vectors.

HOG (Histogram of Oriented Gradients)

A feature descriptor technique used in dlib's face detector. It analyzes the gradient direction and magnitude in small regions of an image. Faces have characteristic gradient patterns (edges around eyes, nose, mouth), which HOG detects without needing a neural network. It's fast and reliable for frontal faces.

Supabase

An open-source Backend-as-a-Service (BaaS) platform -- a Firebase alternative built on PostgreSQL. It provides a database, authentication, storage, and REST APIs out of the box. In VIBE ID, it stores all student/teacher/subject/attendance data. The Python SDK lets you query it with a chain-style syntax similar to SQL.

BaaS (Backend as a Service)

A cloud service that provides ready-made backend infrastructure (database, auth, file storage, APIs) so developers can build apps without managing servers. Supabase is a BaaS. Instead of writing SQL queries and setting up servers, you use the SDK to interact with a hosted database.

Glassmorphism

A UI design trend featuring semi-transparent, blurred backgrounds that create a "frosted glass" effect. Achieved with CSS backdrop-filter: blur() and transparent backgrounds. In VIBE ID, the home page portal cards use glassmorphism to create depth against the dark aurora background.

> [Bulb] You've reached the end! This guide covers every file and every significant code block in the VIBE ID project. The key to understanding the full system is seeing how data flows: Camera -> Face Pipeline -> Database -> Attendance Report, and how Streamlit's session state + rerun model ties everything together into an interactive web app.